

IIO 的 fops / read_raw 在内核哪里

—— 源码实锤:为什么你的驱动里看不到 file_operations

基于 Linux 5.4 内核源码 drivers/iio/industrialio-core.c

MPU-6050 IIO 驱动学习 · 2026 年 9 月

一、问题

在经典字符设备驱动(如 LED)里,你要亲手写 `file_operations (.open / .read / .write)`、`register_chrdev`、建 `/dev` 节点。但在我们的 MPU-6050 IIO 驱动里,整份代码看不到一个 `file_operations`,只写了 `read_raw` 和通道数组。这是为什么?数据到底怎么从用户的 `cat` 流到你的芯片?

答案:**fops 不是不存在,而是 IIO 子系统替你写好了,住在内核源码里,不在你的驱动里。**你只需提供"芯片特有的那块回调"(`read_raw`),框架的 `fops/sysfs` 外壳会在需要数据时回调它。

二、调用链全貌

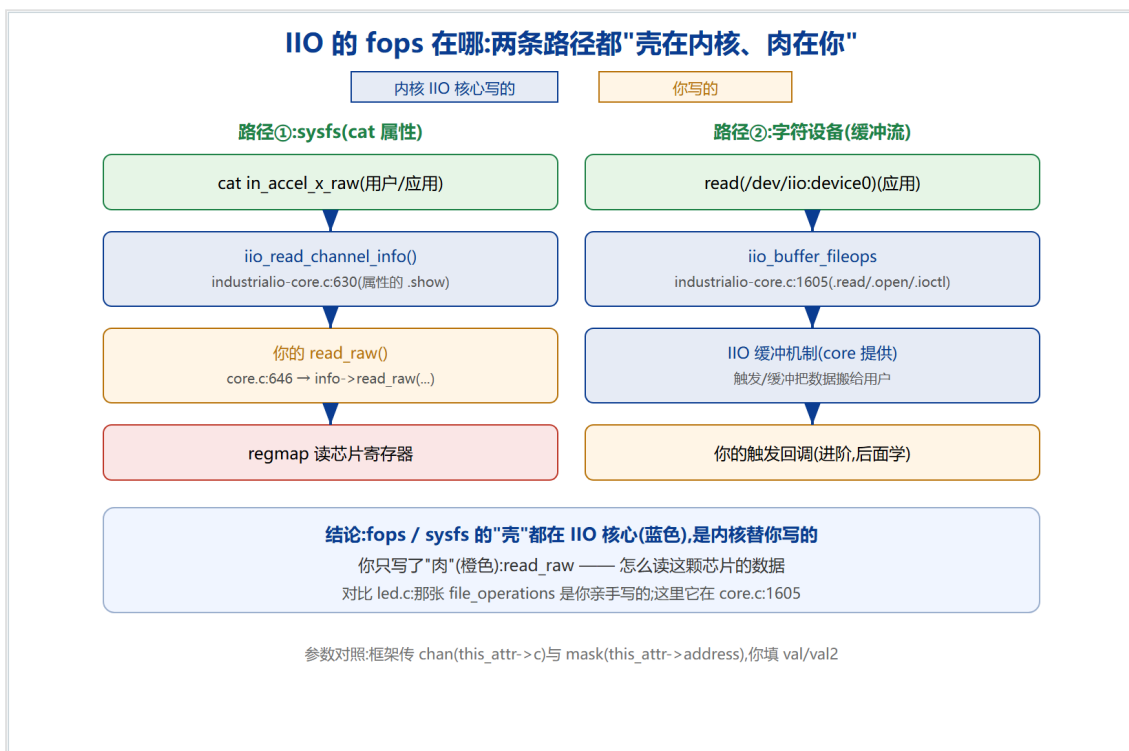


图 1 两条路径:外壳(fops/sysfs)在 IIO 核心,数据回调(read_raw)在你的驱动

三、源码实锤(Linux 5.4, drivers/iio/industrialio-core.c)

3.1 你 cat 属性时,真正执行的是内核的 iio_read_channel_info()

第 630 行起,这个函数是每个通道属性(如 `in_accel_x_raw`)的 `.show` 处理函数——**内核写的**:

```
630 static ssize_t iio_read_channel_info(struct device *dev,
631                                     struct device_attribute *attr, char *buf)
632 {
634     struct iio_dev *indio_dev = dev_to_iio_dev(dev);
635     struct iio_dev_attr *this_attr = to_iio_dev_attr(attr);
636     ...
646     ret = indio_dev->info->read_raw(indio_dev, this_attr->c,
647                                     &vals[0], &vals[1], this_attr->address); // ← 回调【你的】read_raw
637     ...
652     return iio_format_value(buf, ret, val_len, vals); // 把值格式化成字符串给用户
653 }
```

第 646 行的 `indio_dev->info->read_raw(...)` —— **这就是在调用你写的 `read_raw`**。参数正好对上你的函数签名:

框架传入(内核)	你的 <code>read_raw</code> 形参	含义
<code>this_attr->c</code>	<code>chan</code>	哪个通道(accel-X...)
<code>&vals[0] / &vals[1]</code>	<code>val / val2</code>	你把结果写回这里
<code>this_attr->address</code>	<code>mask</code>	要 RAW 还是 SCALE

3.2 iio_read_channel_info 被设成通道属性的 .show

第 1068 行,框架在"自动生成通道属性"时,把上面这个函数挂成了属性的 `.show`:

```
1068     &iio_read_channel_info, // 生成的 in_accel*_raw 等属性的 .show 回调
```

所以链路是: `cat in_accel_x_raw` → 内核 `sysfs` → `iio_read_channel_info (.show)` → 你的 `read_raw`。

3.3 字符设备那套 file_operations(你问的 fops 本体)

第 1605 行, `/dev/iio:deviceN` 的 `file_operations` ——同样是**内核写的**:

```
1605 static const struct file_operations iio_buffer_fileops = {
1606     .read          = iio_buffer_read_first_n_outer_addr,
1607     .release       = iio_chrdev_release,
1608     .open          = iio_chrdev_open,
1609     .poll          = iio_buffer_poll_addr,
1612     .unlocked_ioctl = iio_ioctl,
```

```
1613     .compat_ioctl    = iio_ioctl,  
1614 };
```

.open / .read / .release / .poll / .ioctl —— 这正是你在 LED 驱动里**亲手写**的那种 file_operations 。区别只是:LED 里是你写的;IIO 里它在 `industrialio-core.c:1605` ,内核替你写好了,你一个字没碰。

四、和 LED 手动字符设备对照

	LED 手动字符设备	你的 IIO 驱动
file_operations	你写(.open/.read/.write)	内核写(core.c:1605 iio_buffer_fileops)
sysfs 属性的 .show	—(没用 sysfs)	内核写(core.c:630 iio_read_channel_info)
建 /dev 节点	你 register_chrdev / device_create	IIO 自动建 /dev/iio:deviceN
生成属性文件	—	IIO 按你的通道自动生成 in_accel_*_raw
你真正写的	全部管道	只有 iio_chan_spec + read_raw

五、结论

两条取数据的路径——sysfs(cat 属性)和字符设备(read /dev/iio:deviceN)——它们的"外壳"(fops、sysfs 的 show、建节点、生成属性)全在 IIO 核心里,是内核替你写好的。你的驱动只提供"芯片特有的那一小块": read_raw (怎么读这颗芯片的数据)。框架在需要数据时回调它。

一句话:用子系统 = 继承一套现成的 fops/sysfs 外壳,你只实现"芯片特有的回调"。你代码里看不到 fops,不是没有,是你站在了框架的肩膀上。你的代码越少,越说明框架替你干得越多。

想亲眼验证,在你的内核源码树里看这个文件即可:

```
drivers/iio/industrialio-core.c
:630  iio_read_channel_info()      # sysfs 读,回调你的 read_raw
:646  indio_dev->info->read_raw()  # 回调点
:1068 .show = &iio_read_channel_info
:1605 iio_buffer_fileops          # /dev/iio:deviceN 的 file_operations
```

本文基于 Linux 5.4 内核源码(行号随版本略有差异)。配套仓库:github.com/fyy0619-cell/xiangmu